Campus Events Project Plan

A phased build plan for the campus events app: a Telegram bot that turns posters into events, and a mobile-first web feed with Google login and favorites. The bot (Phase 2) is built before the frontend feed (Phase 3). Each phase has concrete steps, a checkpoint to confirm it works, and the mistakes beginners usually hit.

Phase 0: Setup (about half a day)

Steps

1. Install Python 3.11+, Node 20+, Git, and VS Code.
2. Create accounts: GitHub, Supabase, Google AI Studio (for the Gemini key), and Telegram.
3. Create the project folder with two subfolders: `bot/` (Python) and `web/` (Next.js, created in Phase 3), inside `campus-events/`.
4. Run `git init`, and add a `.gitignore` containing `.env`, `venv/`, and `node_modules/` before your first commit.

Checkpoint: `python --version`, `node --version`, and `git --version` all print versions.

Pitfall: committing API keys to GitHub. Set up `.gitignore` first.

Phase 1: Database (about half a day)

Steps

1. Create a Supabase project and save the database password.
2. In the SQL Editor, run the schema: `clubs`, `events`, `saved_events`, plus the indexes.
3. Add the `users` trigger that copies new `auth.users` rows into `public.users`.
4. Turn on RLS for every table and add the policies:
   - `events`: public `SELECT` only. Writes happen through the service role key, which bypasses RLS.
   - `saved_events`: `SELECT`, `INSERT`, and `DELETE` where `auth.uid() = user_id`.
   - `clubs`: no public policies, so only the bot can read it.
5. Create a **public** Storage bucket named `posters`.

6. Collect three values for your `.env`: the project URL, the `anon` key (frontend), and the `service_role` key (bot only).
7. Insert 3 fake events by hand in the Table Editor.

Checkpoint: open `https://<project>.supabase.co/rest/v1/events?select=*` with the `anon` key in the `apikey` header (or use the Supabase API docs page) and see your fake events as JSON.

Pitfall: forgetting to enable RLS, or enabling it without any policy, so reads return empty.

Phase 2: The Telegram bot (about 3 to 5 days)

Build this in five small steps. Each one runs on its own before you add the next.

2a. Create the bot (15 min)

- Message @BotFather, send `/newbot`, pick a name, and copy the token into `bot/.env`.
- Create a virtual environment and install packages: `python-telegram-bot`, `google-genai`, `supabase`, `python-dotenv`, `pillow`.

2b. Echo bot

- Write about 15 lines that reply to any text message with the same text, using polling.
- *Checkpoint:* you message the bot and it answers.
- This teaches the core pattern: handler, async function, `application.run_polling()`.

2c. Receive and download a photo

- Add a handler for photos and for image documents (`filters.PHOTO | filters.Document.IMAGE`).
- Download the largest photo into memory as bytes.
- *Checkpoint:* the bot replies "Got your poster, 842 KB".

2d. Extract with Gemini

- Send the bytes plus the prompt to Gemini with `response_mime_type="application/json"`.
- Ask for `title`, `club_name`, `event_start`, `event_end`, `location`, and `rsvp_link` (ISO 8601 with `+05:30`, `null` when missing).
- Parse the JSON, apply the `event_end = event_start + 3h` default, and print the result.

- Test with 10 to 15 real posters, including messy ones, and refine the prompt until most come out right.
- *Checkpoint:* sending a poster prints correct JSON in your terminal.

2e. Save to Supabase

- Check the sender against the `clubs` allowlist. Ignore anyone who isn't in it.
- Upload the image to the `posters` bucket and build the public URL.
- Reply with the parsed details and **Confirm / Cancel** inline buttons. Insert into `events` only on Confirm.
- Add a `/delete` command so a club can remove its own last event.
- *Checkpoint:* you send a poster, tap Confirm, and a new row appears in Supabase.

Pitfalls

- The bot being silent usually means an unhandled exception, so turn on `logging`.
- Gemini sometimes returns a wrong year. Validate that `event_start` is in the future and within about 1 year, and ask the user to fix it if not.
- Use the service role key only in `bot/.env`.

Phase 3: Frontend feed (about 1 to 2 weeks, mostly your Figma translation)

Steps

1. Scaffold with `npx create-next-app@latest web` (TypeScript, Tailwind, App Router).
2. Install `@supabase/supabase-js` and create a single Supabase client file using the `anon` key.
3. Break the Figma design into components: `EventCard`, `EventFeed`, `Header`, and loading/empty states.
4. Fetch events with `.from('events').select('*').gt('event_end', new Date().toISOString()).order('event_start')`.
5. Render the cards, using Next's `<Image>` for posters (allow your Supabase domain in `next.config`).
6. Handle loading, empty, and error states. Format dates in IST with `Intl.DateTimeFormat`.
7. Check the layout at 375px width first, since this is mobile-first.

Checkpoint: a poster sent to the bot appears in the feed after a refresh.

Pitfall: timezone bugs. Store UTC, display IST, and never format dates by hand.

Phase 4: Auth and favorites (about 1 week)

Steps

1. In Google Cloud Console, create an OAuth client and add Supabase's callback URL. Enable the Google provider in Supabase.
2. Build an `AuthProvider` context that exposes `session` and `signIn()`.
3. Add the heart button with optimistic UI: toggle immediately, `INSERT` or `DELETE` in the background, roll back with a toast on failure.
4. Pending favorite: if `session === null`, save `event_id` to localStorage and start OAuth. After login, read it, save the event, and clear it.
5. Load the user's saved IDs once after login so hearts render correctly.
6. Add a "Saved" page that shows only the user's favorites.

Checkpoint: log out, tap a heart, sign in with Google, and land back with that event saved.

Pitfall: redirect URL mismatches between Google, Supabase, and localhost versus production. Add both URLs.

Phase 5: Deploy (about 2 to 3 days)

1. **Frontend:** push to GitHub, import into Vercel, add the two public env vars, and add the production URL to Supabase's auth redirect list.
2. **Bot:** host on Railway, Render, or Fly.io as a worker process with a start command like `python bot.py` and the env vars set. Only one instance can poll at a time, so stop your local bot first.
3. **Cleanup:** schedule `pg_cron` weekly to delete events where `event_end < now() - interval '30 days'`.
4. **Final tests:** run the full flow on a real phone: send a poster, confirm, see it in the feed, log in, and favorite it.

Rough total

About 4 to 6 weeks part-time for a beginner. The Figma-to-code work in Phase 3 is the biggest variable.